Dossier GitOps & CI/CD

Nom et prénom : DOOM Tevaitea

Date : 6 novembre 2026

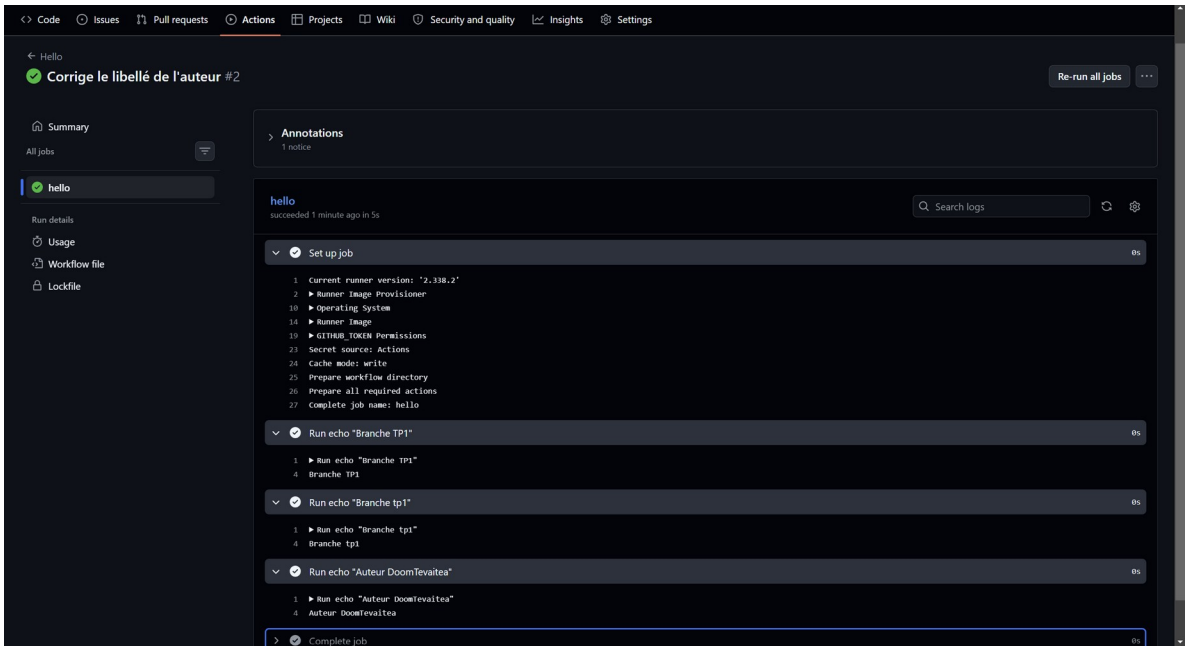
Dépôt GitHub : github.com/DoomTevaitea/tasks-api

TP1 : première CI avec GitHub Actions

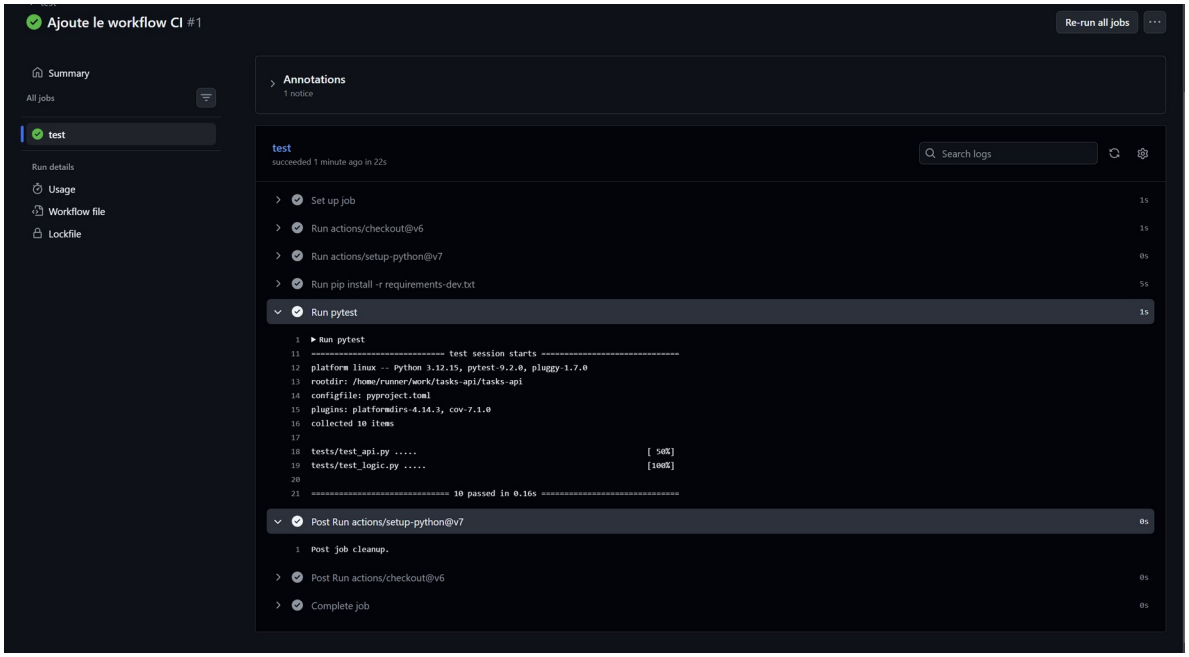
Objectif

Ce TP m'a fait mettre en place une intégration continue sur mon dépôt `tasks-api`. Chaque `git push` lance maintenant les tests automatiquement sur une machine neuve. J'ai aussi appris à lire un run en échec et à ne fusionner une pull request que lorsque la CI est verte.

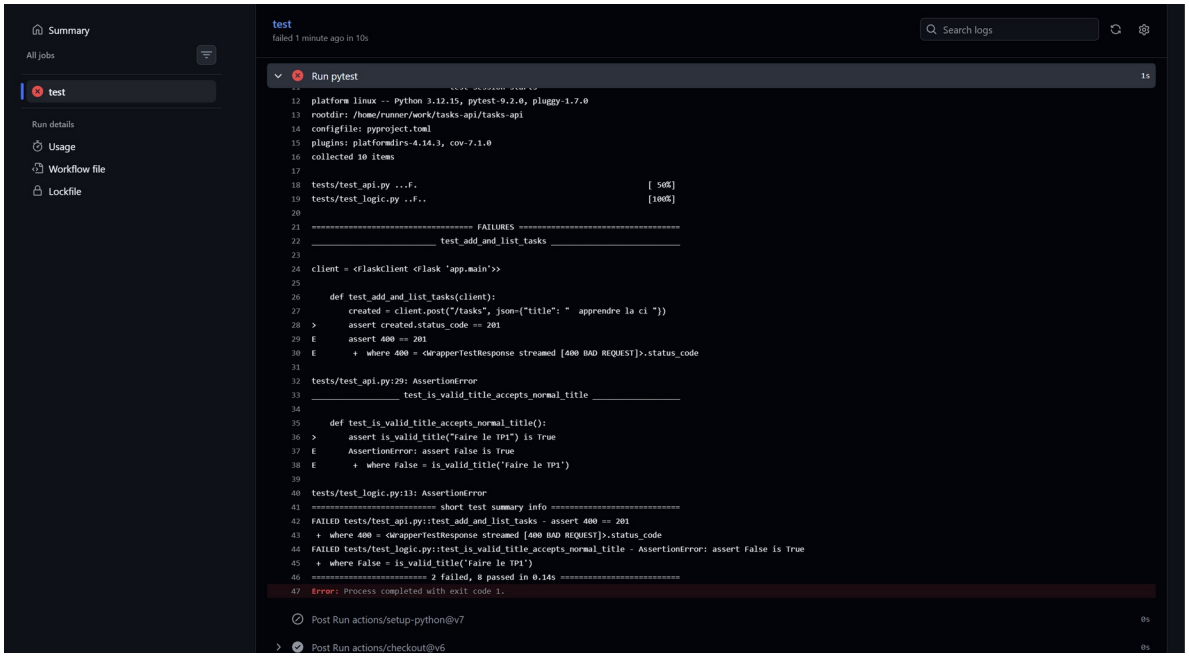
Captures d'écran



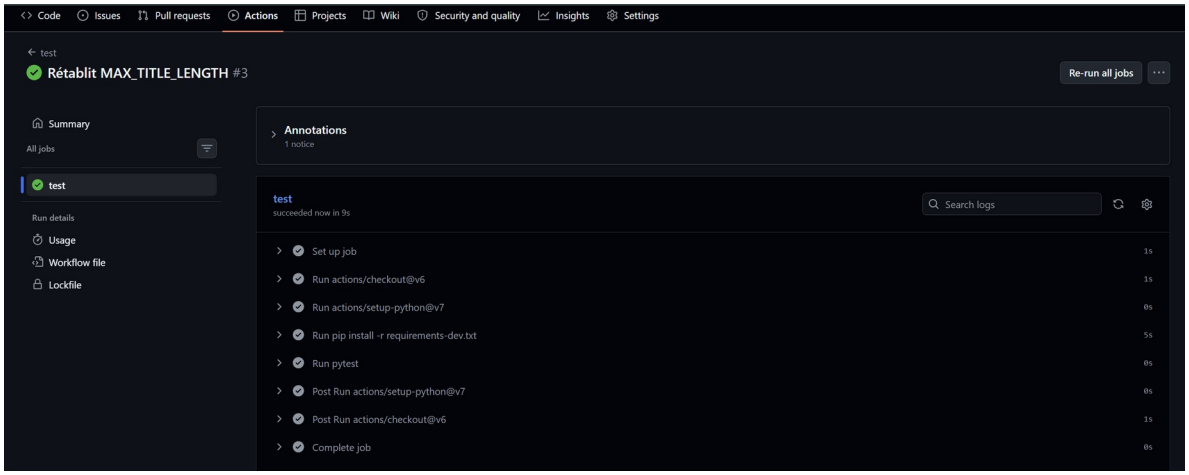
TP1-1 : le run « Hello » a réussi, et l'étape dépliée affiche la branche `tp1` et l'auteur `DoomTevaitea`.



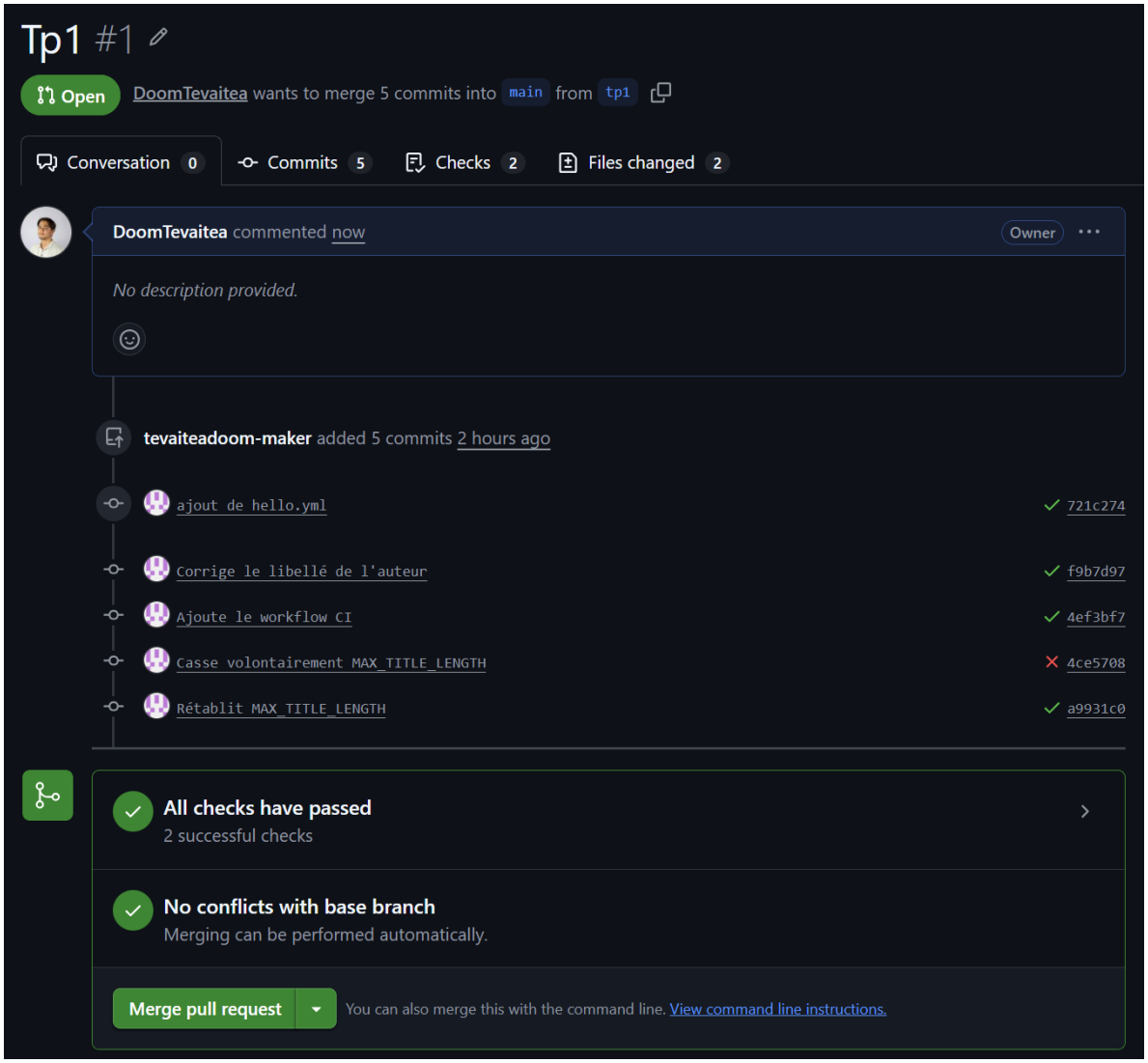
TP1-2 : le run du workflow `ci.yml` a réussi, et on voit ses étapes : récupération du code, installation de Python, installation des dépendances et lancement de `pytest`, qui indique `10 passed`.



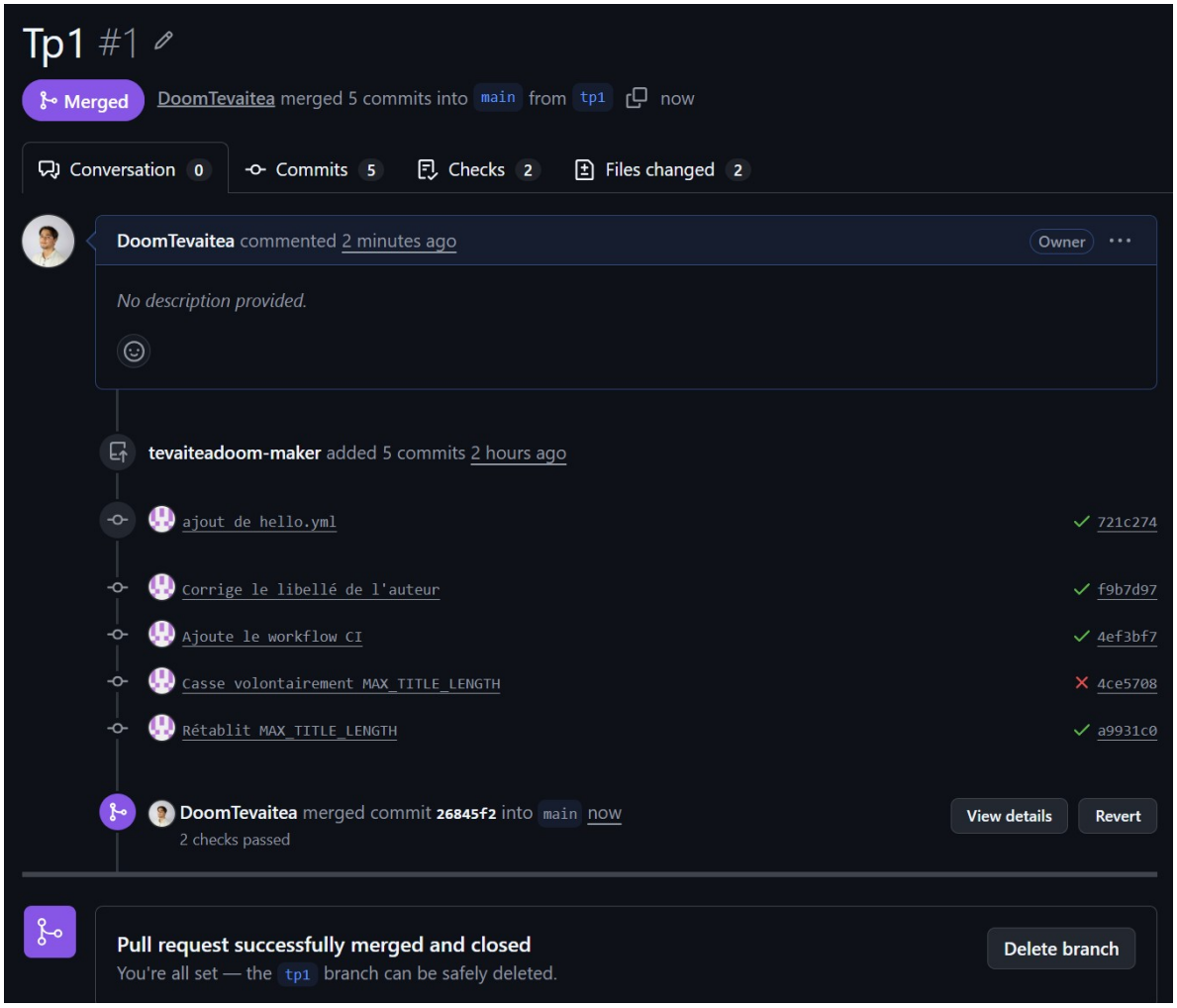
TP1-3 : après la modification volontaire de `MAX_TITLE_LENGTH`, le run est rouge : l'étape `pytest` dépliée indique 2 tests en échec et 8 réussis.



TP1-4 : après le rétablissement de la valeur d'origine, le run suivant redevient vert.



TP1-5 : la pull request de `tp1` vers `main` affiche ses vérifications vertes, avant la fusion.



TP1-6 : la pull request est fusionnée dans `main`.



TP1-7 : le README.md affiché sur GitHub montre le badge de statut de la CI, avec la mention `passing`.

Réponses aux questions

1. Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Le run a duré 10 secondes, dont 5 secondes pour le job `hello`. Pour voir les logs, on ouvre l'onglet Actions, puis le run et le job `hello`, et on déplie l'étape voulue.

2. Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Deux tests ont échoué : `test_add_and_list_tasks`, à la ligne 29 de `tests/test_api.py`, et `test_is_valid_title_accepts_normal_title`, à la ligne 13 de `tests/test_logic.py`. Avec une limite de 5 caractères, les titres utilisés par ces tests étaient refusés.

Le log de l'étape `pytest` m'a permis de les trouver rapidement : le résumé final liste chaque test en échec, et chaque erreur indique le fichier, la ligne et la valeur obtenue.

3. Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

Une CI verte montre que les tests passent sur le code proposé. Attendre ce résultat évite d'introduire dans `main` un changement qui casse l'application et bloquerait le reste de l'équipe.

Difficultés rencontrées

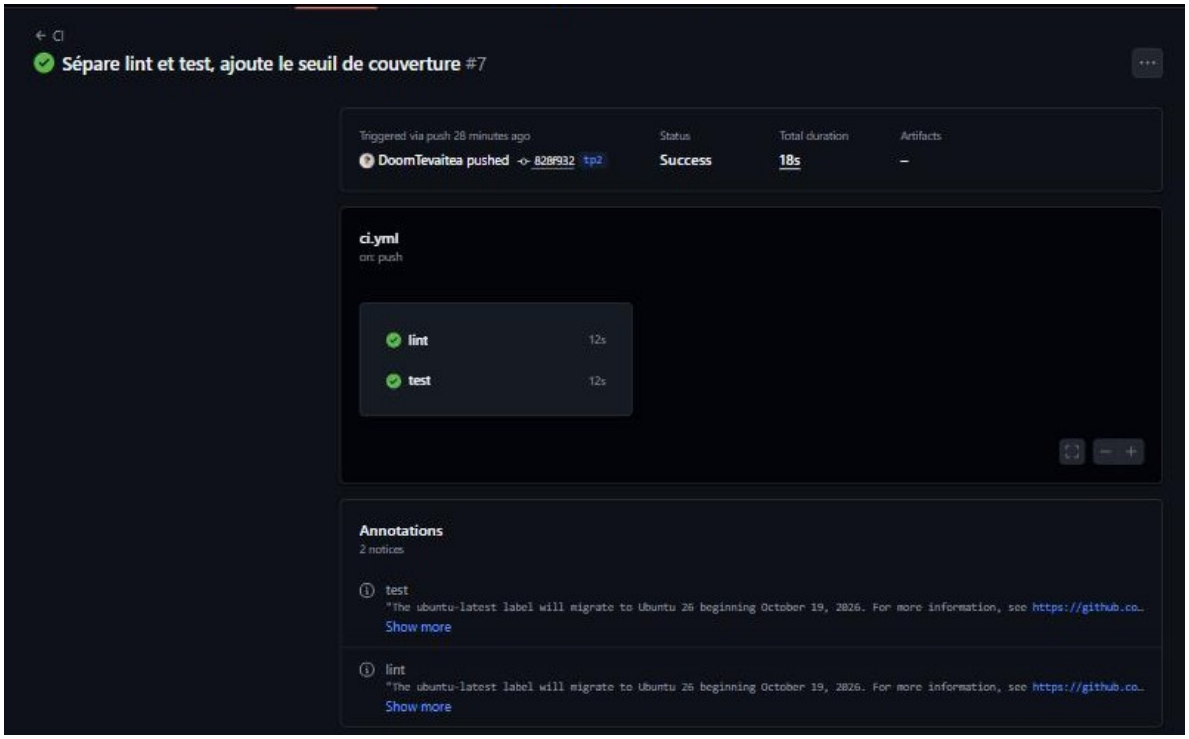
Difficulté	Solution
Le script de vérification était introuvable, à cause d'une faute dans le nom du dossier.	J'ai recherché le fichier sur le disque pour retrouver son chemin exact.
Le premier `git push` a été refusé, car la branche n'avait pas de branche distante.	J'ai utilisé `git push -u origin tp1`.
Mon premier `hello.yml` était mal structuré : étapes sur une seule ligne et `${{ }}` autour d'un simple texte.	J'ai mis une étape par ligne, avec un tiret, et j'ai gardé `${{ }}` pour les valeurs de GitHub.
Le bloc `with:` de `ci.yml` était mal aligné.	Je l'ai aligné sur `uses` et j'ai ajouté `python-version: "3.12"`.
`git add` ne trouvait pas le fichier, car mon terminal était déjà dans `.github/workflows`.	J'ai utilisé le bon chemin, puis je suis revenu à la racine du dépôt.
Mon dépôt se trouvait dans un sous-dossier `tasks-api\tasks-api`.	J'ai repéré le problème avec `ls`, puis je suis allé dans le bon dossier.

TP2 : une CI solide

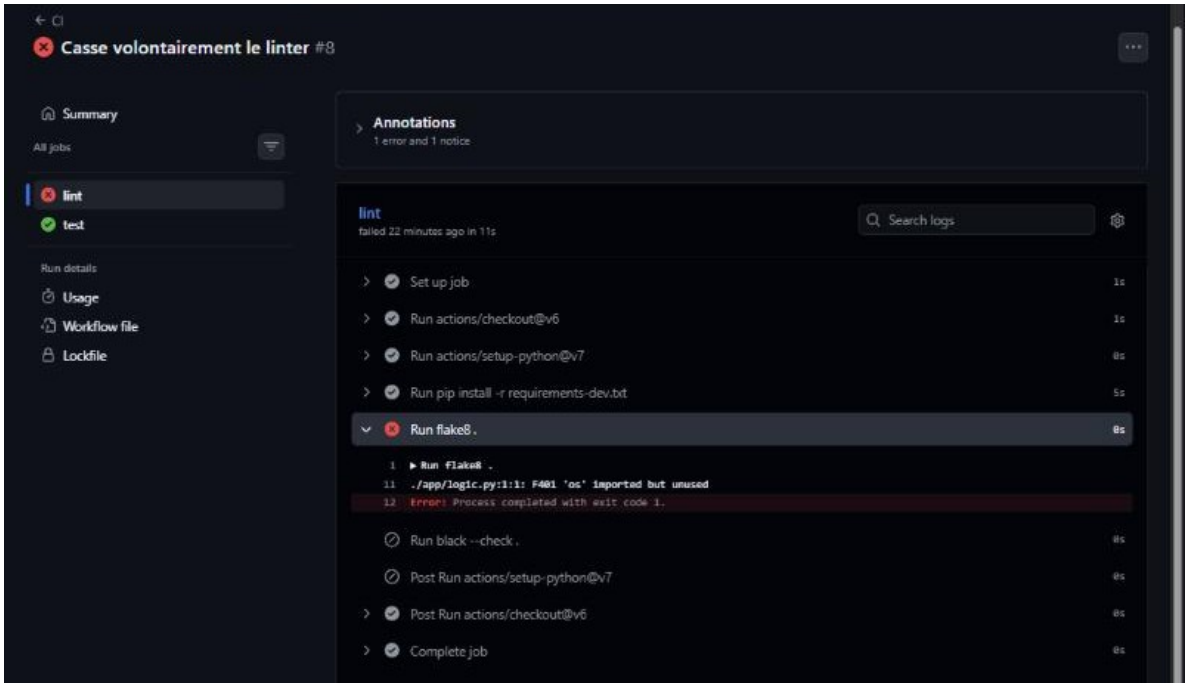
Objectif

Ce TP a renforcé ma CI avec un contrôle de qualité du code, un seuil de couverture et des tests sur trois versions de Python. J'ai ajouté un cache, un artefact, la protection de `main`, un secret dans un workflow, et j'ai résolu un conflit et trois pipelines cassés.

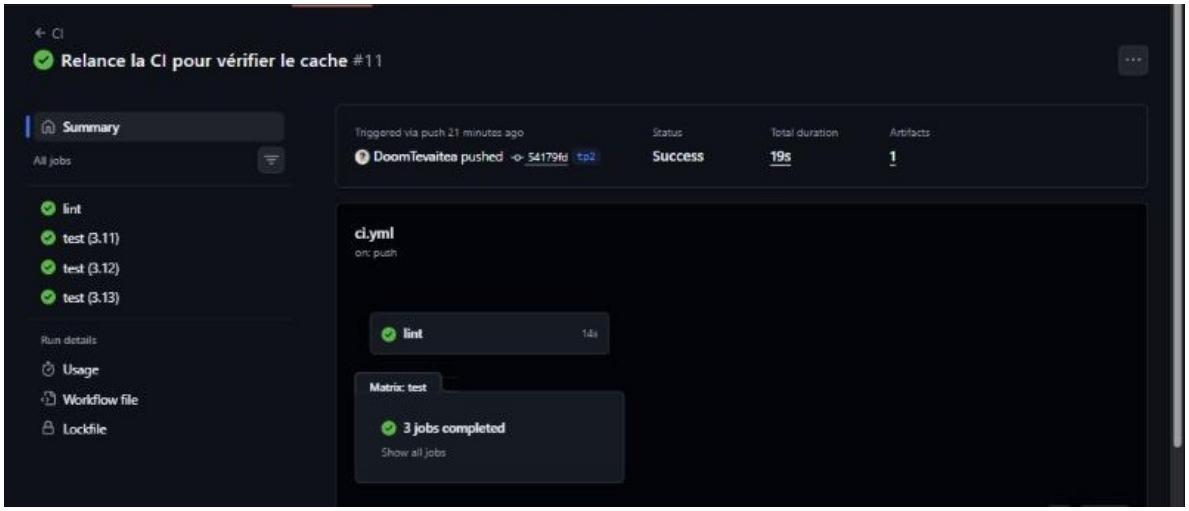
Captures d'écran



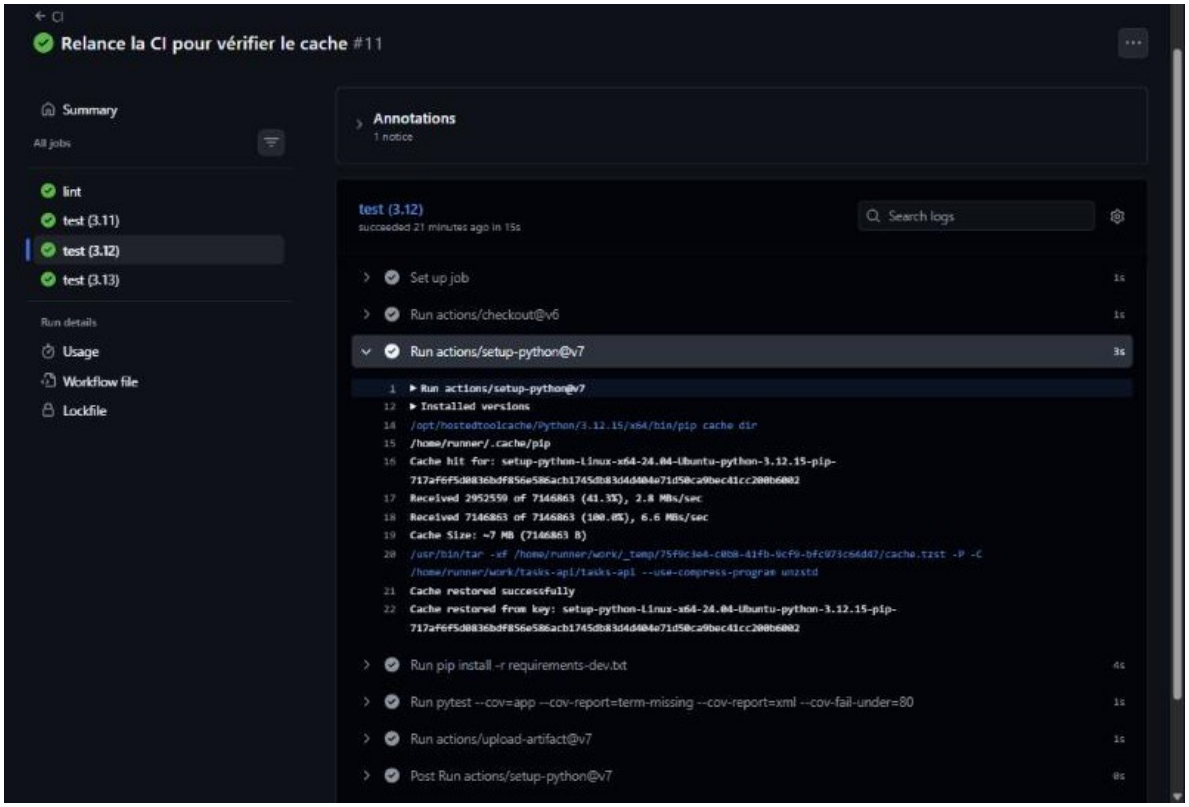
TP2-1 : le run est vert et montre les deux jobs `lint` et `test`.



TP2-2 : le job `lint` est rouge : flake8 signale `F401`, car `os` est importé sans être utilisé.



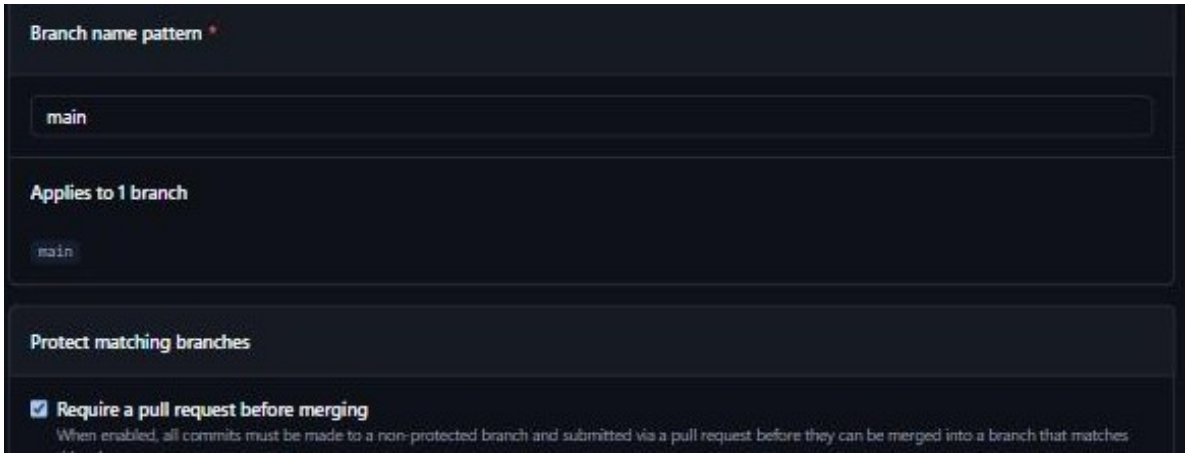
TP2-3 : la colonne de gauche montre les jobs `test (3.11)`, `test (3.12)` et `test (3.13)` de la matrice.



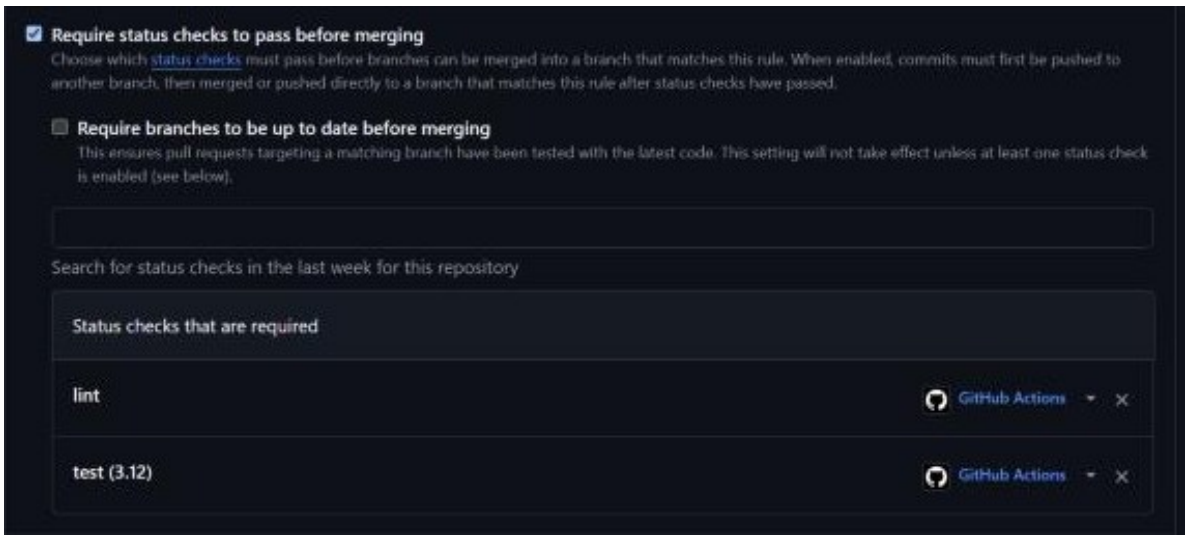
TP2-4 (cache) : au deuxième run, l'étape `setup-python` indique que le cache a été restauré.



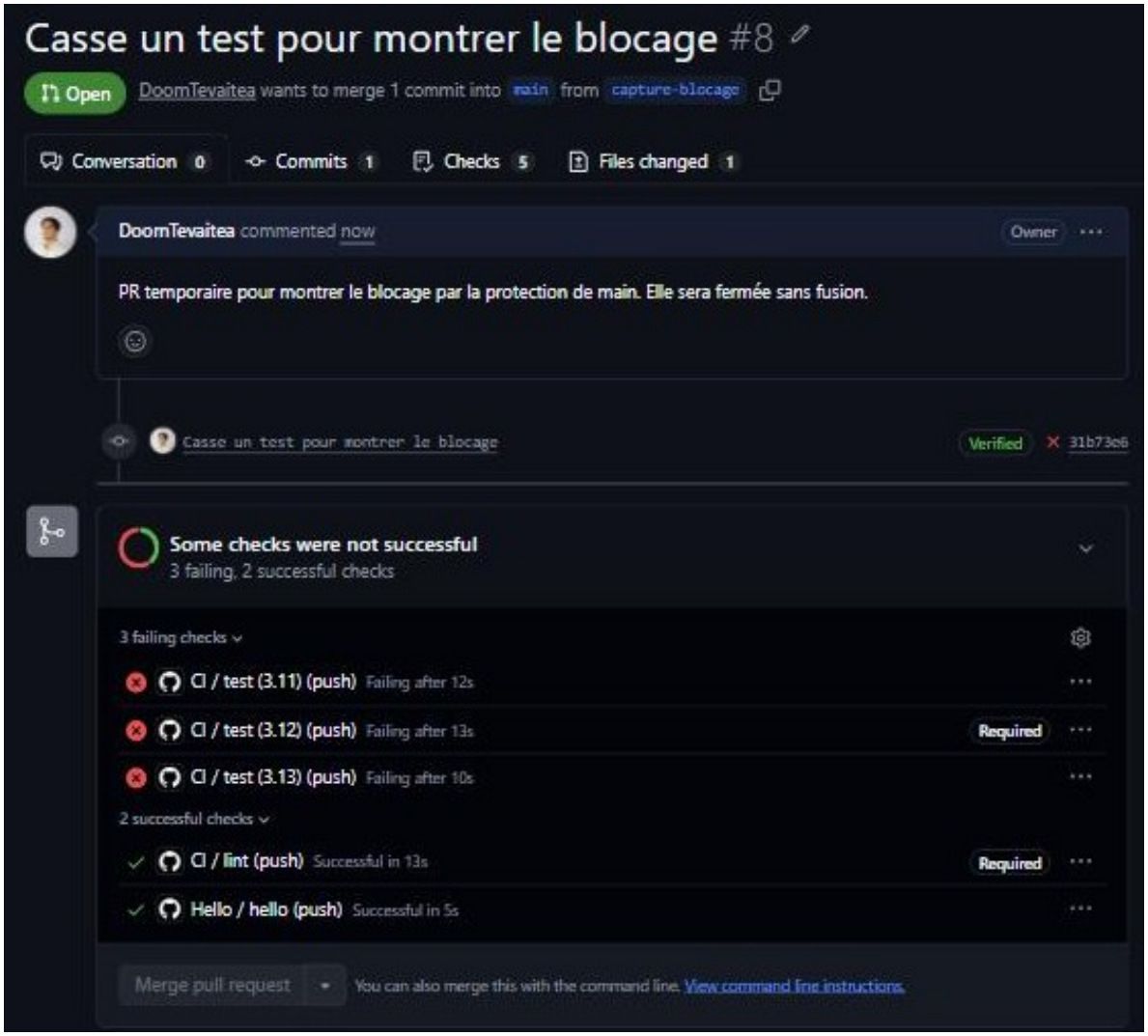
TP2-4 (artefact) : la section Artifacts de la page du run contient `coverage-report`.



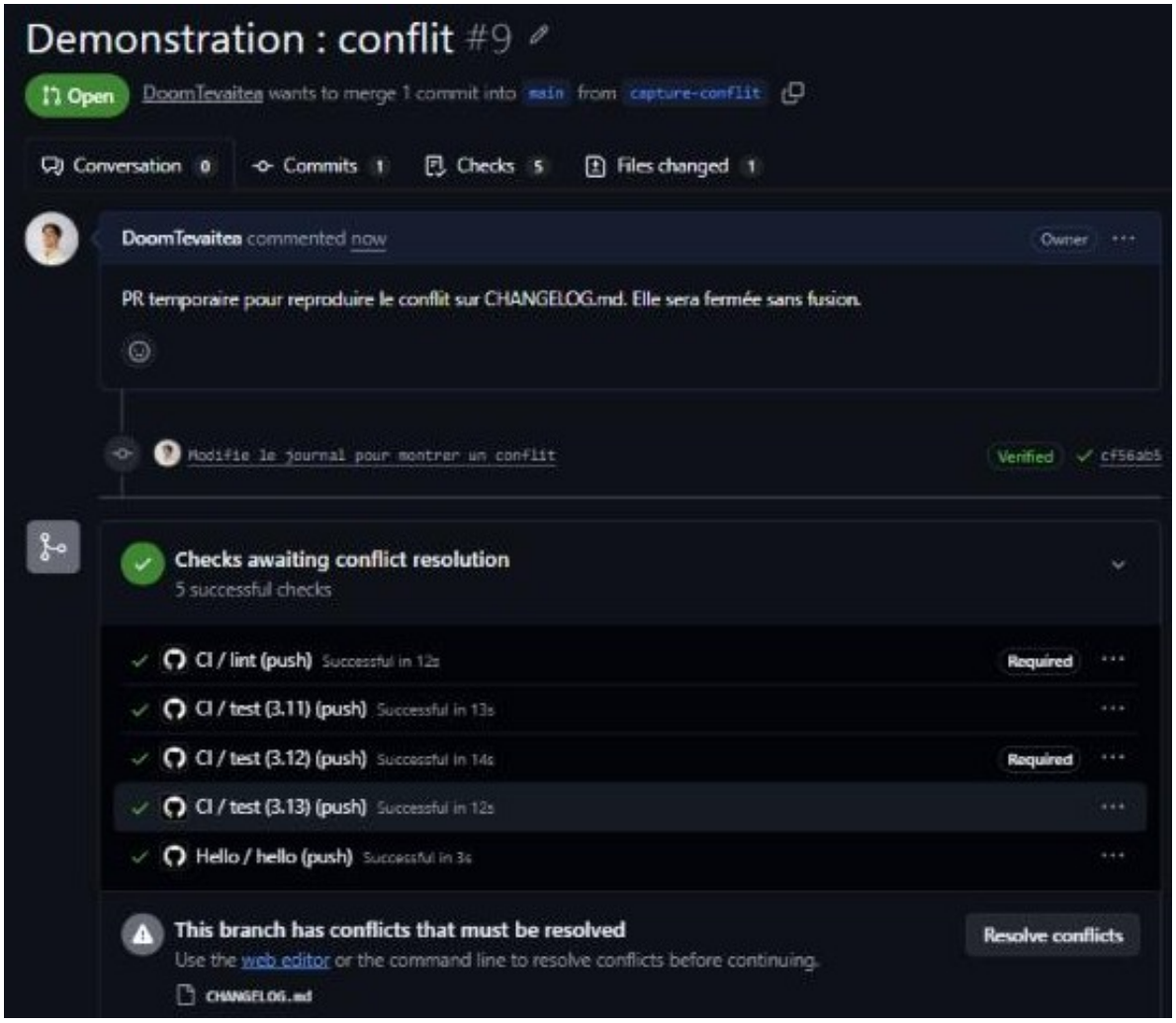
TP2-5 (pull request) : la règle de protection de `main` exige une pull request avant toute fusion.



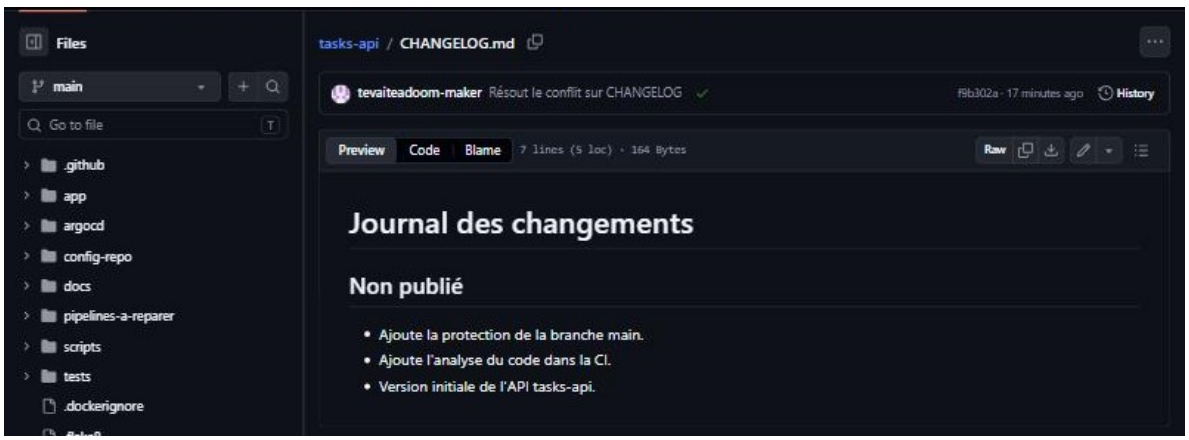
TP2-5 (vérifications) : la même règle impose les vérifications `lint` et `test (3.12)`, sans contournement possible.



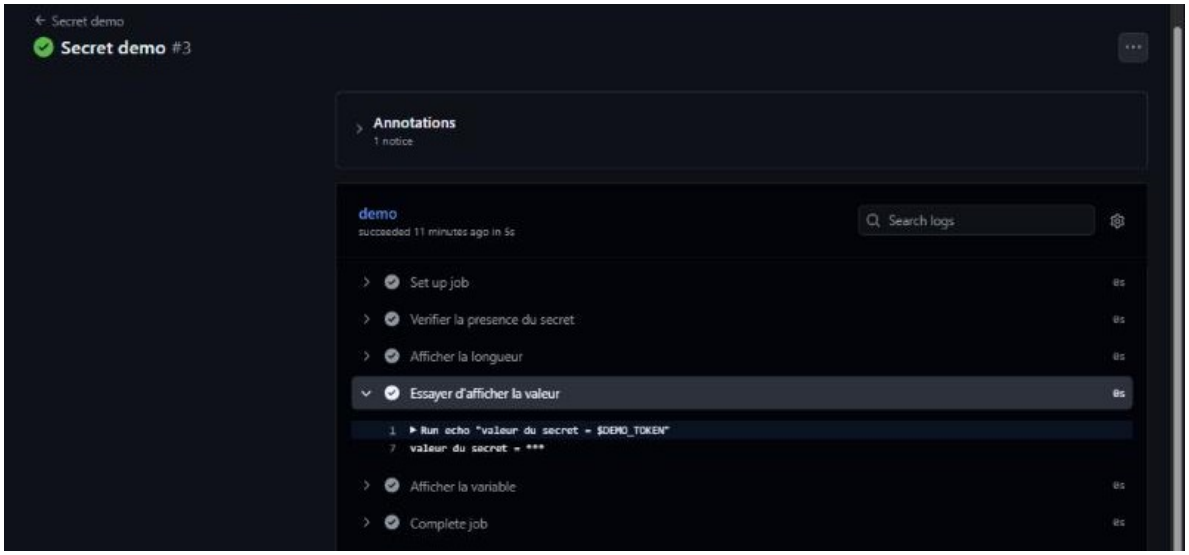
TP2-6 : dans cette pull request de démonstration, le bouton de fusion est désactivé, car la vérification obligatoire `test (3.12)` est en échec.



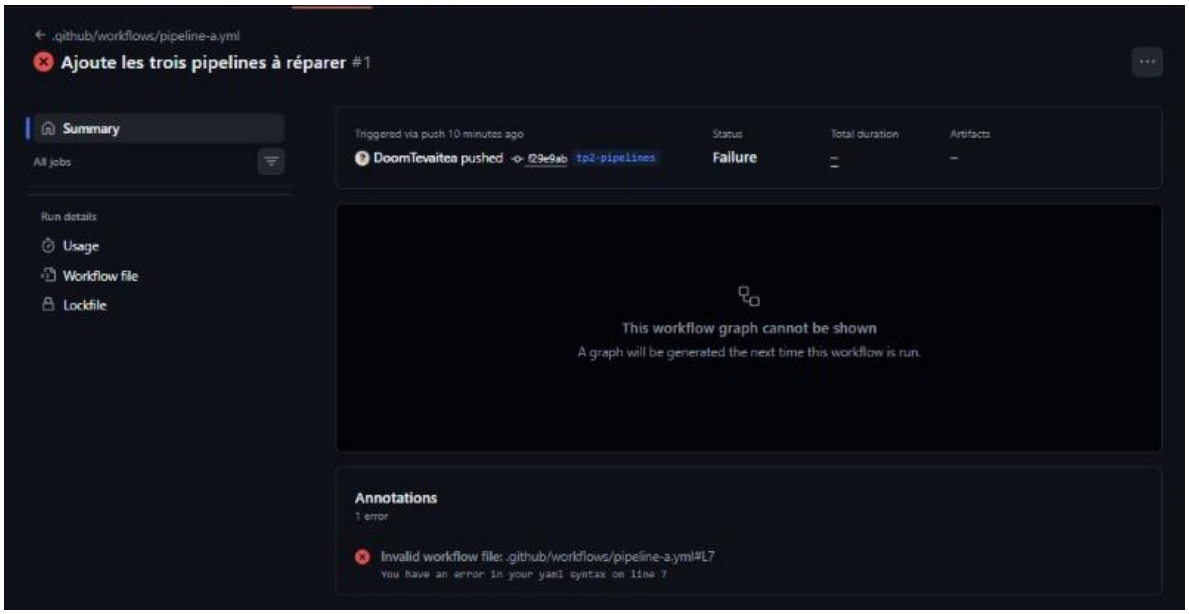
TP2-7 : cette pull request de démonstration reproduit le conflit sur `CHANGELOG.md`, signalé par GitHub.



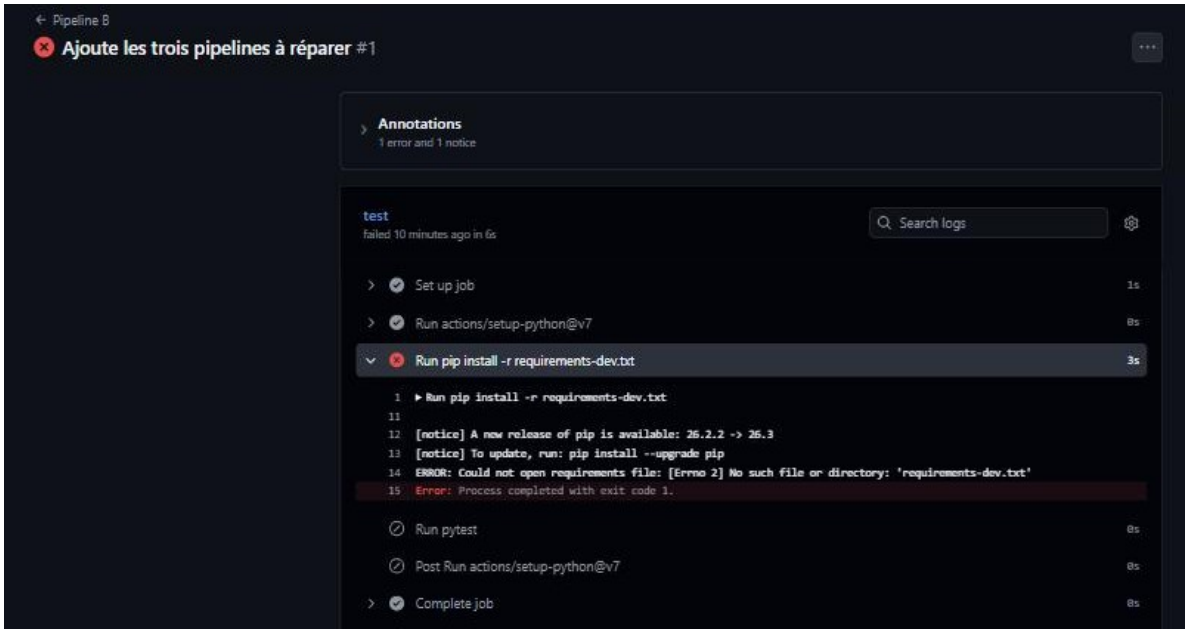
TP2-8 : le fichier `CHANGELOG.md` final sur `main` contient les deux lignes.



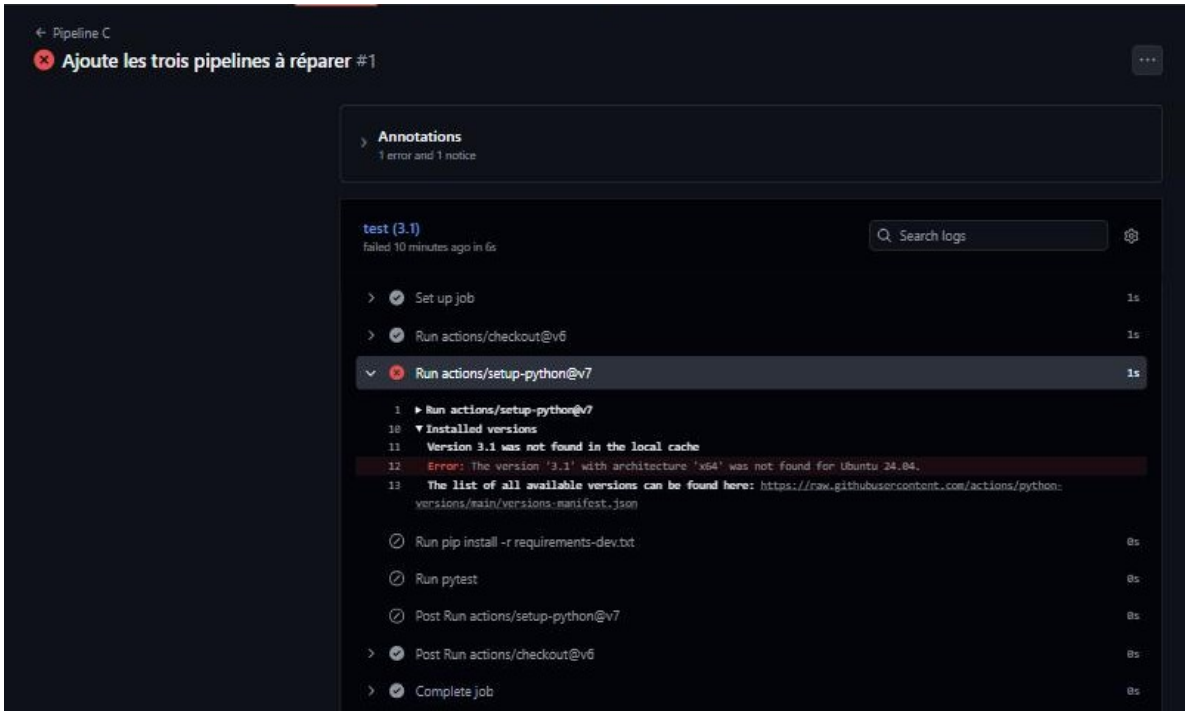
TP2-9 : dans les logs, la valeur du secret est remplacée par ***.



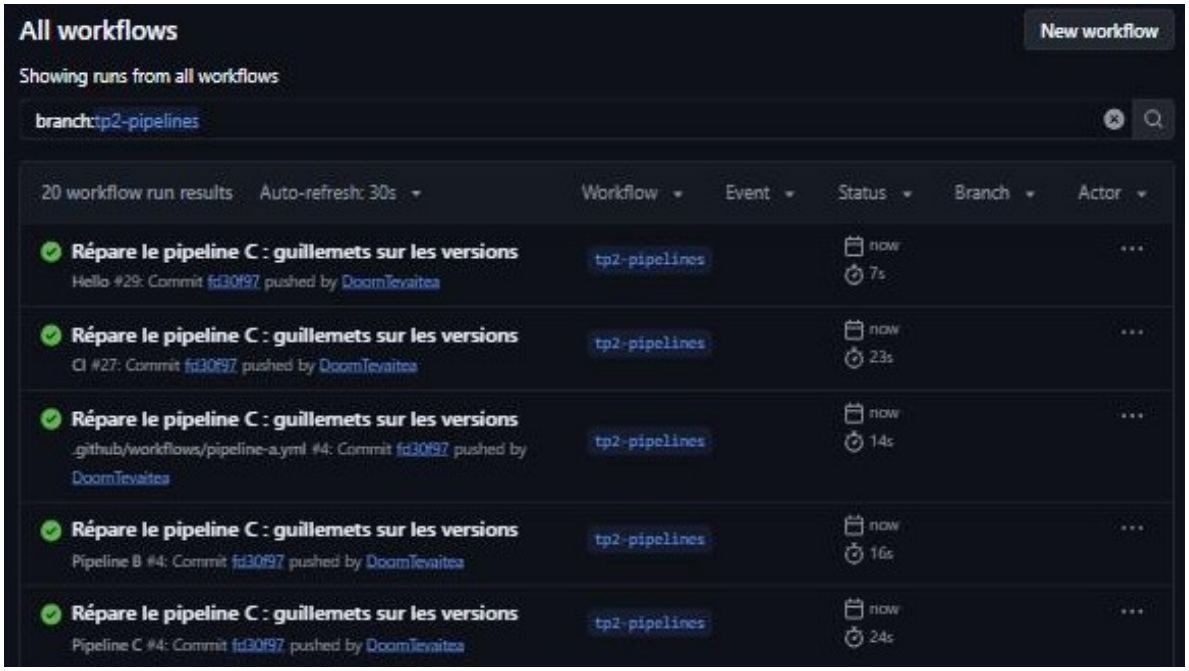
TP2-10A : Pipeline A échoue dès le départ : GitHub signale une erreur de syntaxe YAML à la ligne 7.



TP2-10B : Pipeline B échoue à l'installation, car `requirements-dev.txt` est introuvable.



TP2-10C : Pipeline C échoue, car la version `3.10` a été lue comme `3.1`.



TP2-11 : la liste des runs montre les trois pipelines réparés au vert.

Réponses aux questions

Quelle différence entre un linter (flake8) et un formateur (black) ?

Le linter signale des erreurs et des mauvaises pratiques sans modifier le code. Le formateur impose un style unique, et `black --check` vérifie ce style sans rien réécrire.

Pourquoi séparer lint et test en deux jobs ?

On voit tout de suite si le problème vient de la qualité du code ou du comportement de l'application. Les deux jobs peuvent aussi tourner en parallèle.

Pourquoi tester sur plusieurs versions de Python ?

Pour vérifier que le code fonctionne sur les versions réellement utilisées, et repérer une incompatibilité avant la production.

Que gagne-t-on avec le cache ?

Les dépendances ne sont pas retéléchargées à chaque run, donc l'installation est plus rapide.

Un artefact, c'est quoi ?

C'est un fichier produit par un run et conservé après celui-ci, ici le rapport de couverture. On peut le télécharger depuis la page du run.

Pourquoi interdire de pousser directement sur main ?

Tout changement passe ainsi par une pull request et par les vérifications. `main` ne reçoit donc que du code contrôlé.

Pourquoi Git ne peut-il pas résoudre ce conflit seul ?

Deux branches ont modifié le même endroit du fichier, et Git ne sait pas quelle version garder. Une personne doit décider.

Qu'est-ce qui a vérifié le code après la résolution ?

La CI s'est relancée sur le commit de résolution, et la pull request n'a été fusionnée qu'une fois ses vérifications vertes.

Pourquoi GitHub masque-t-il le secret ?

Pour éviter que sa valeur apparaisse dans les logs, que d'autres personnes peuvent lire. Ce masquage reste une protection limitée.

Que se passe-t-il si on le commite dans le dépôt ?

Il reste dans l'historique Git, même après suppression du fichier. Il est alors compromis : il faut le révoquer et en créer un nouveau.

Secret ou variable : comment choisir ?

Une valeur sensible, comme un mot de passe ou un jeton, va dans un secret. Une valeur non sensible, comme un nom d'environnement, va dans une variable.

Pipeline A : quel bug, comment le trouver, comment l'éviter ?

Un espace de trop devant `-  run` rendait le YAML invalide. GitHub indiquait l'erreur de syntaxe à la ligne 7. Pour l'éviter, on vérifie l'alignement ou on utilise un validateur YAML.

Pipeline B : quel bug, comment le trouver, comment l'éviter ?

Il manquait `actions/checkout`, donc le fichier de dépendances était absent. Le log de `pip install` indiquait le fichier introuvable. Il faut se rappeler que la machine démarre vide.

Pipeline C : quel bug, comment le trouver, comment l'éviter ?

Sans guillemets, `3.10` était lu comme `3.1`. Le log et le nom du job `test (3.1)` le montraient. Il faut toujours mettre les versions entre guillemets.

Difficultés rencontrées

Difficulté	Solution
Mon workflow `secret-demo.yml` ne se lançait pas, car un deux-points suivi d'un espace rendait son YAML invalide.	J'ai remplacé ce caractère dans les deux lignes, puis j'ai fusionné la correction par une pull request.
Une fois `main` protégée, je ne pouvais plus la modifier directement.	Chaque correction est passée par une branche et une pull request.
Le bouton Run workflow n'est utilisable qu'avec un workflow valide présent sur `main`.	J'ai fusionné le fichier sur `main` avant de le lancer.
Les états de la partie 3 et de la partie 4 n'étaient plus visibles après la fusion, et je n'avais pas pris ces captures à temps.	Je les ai reproduits avec deux pull requests de démonstration, fermées sans fusion.

TP3 : Docker et livraison continue

Objectif

Ce TP a mis l'application dans une image Docker publiée automatiquement sur GHCR. J'ai créé des versions avec des tags Git, puis un déploiement simulé sur staging et sur production, avec une approbation humaine avant la production.

Captures d'écran

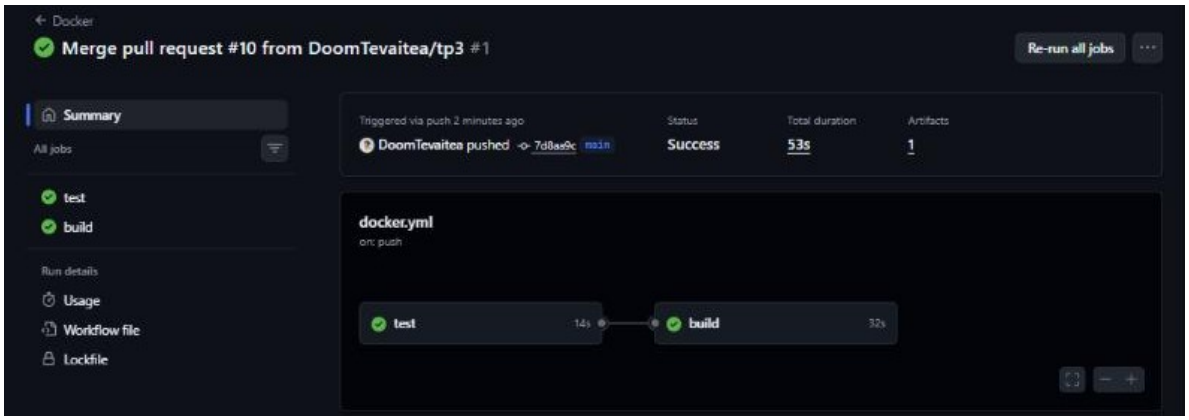
```
PS> docker build --progress=plain -t tasks-api:local .
#11 exporting manifest list sha256:a582518e2ba3f866161d8df96361c815b45ee552ee8fa1ccd1ebbe10055c8261
#11 exporting manifest list sha256:a582518e2ba3f866161d8df96361c815b45ee552ee8fa1ccd1ebbe10055c8261 0.0s done
#11 naming to docker.io/library/tasks-api:local done
#11 unpacking to docker.io/library/tasks-api:local
#11 unpacking to docker.io/library/tasks-api:local 4.7s done
#11 DONE 7.3s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/pj9l6feyzfjiy9ldodho6b64c
PS> curl.exe http://localhost:8000/health
{"status":"ok"}
PS> curl.exe http://localhost:8000/version
{"version":"dev"}
```

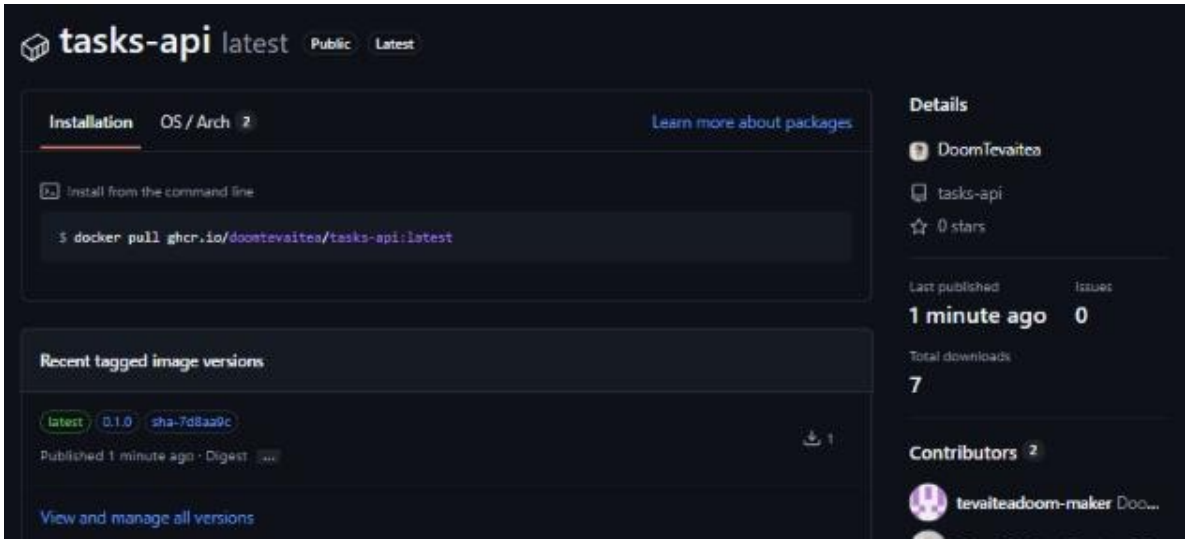
TP3-1 : le `docker build` a réussi, et `/health` répond `{"status":"ok"}`.

```
PS> docker compose up --build -d
 Container tasks-api-api-1 Creating
 Container tasks-api-api-1 Created
 Container tasks-api-api-1 Starting
 Container tasks-api-api-1 Started
PS> docker compose ps
NAME              IMAGE           COMMAND                  SERVICE   CREATED         STATUS                            P
ORTS
tasks-api-api-1   tasks-api-api   "gunicorn --bind 0.0…"   api       3 seconds ago   Up 2 seconds (health: starting)   0
.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
PS> curl.exe http://localhost:8000/version
{"version":"local"}
```

TP3-2 : le service lancé avec `docker compose up --build -d` est en marche, et `/version` répond `local`.



TP3-3 : le run est vert : le job `build` s'exécute après le job `test`.



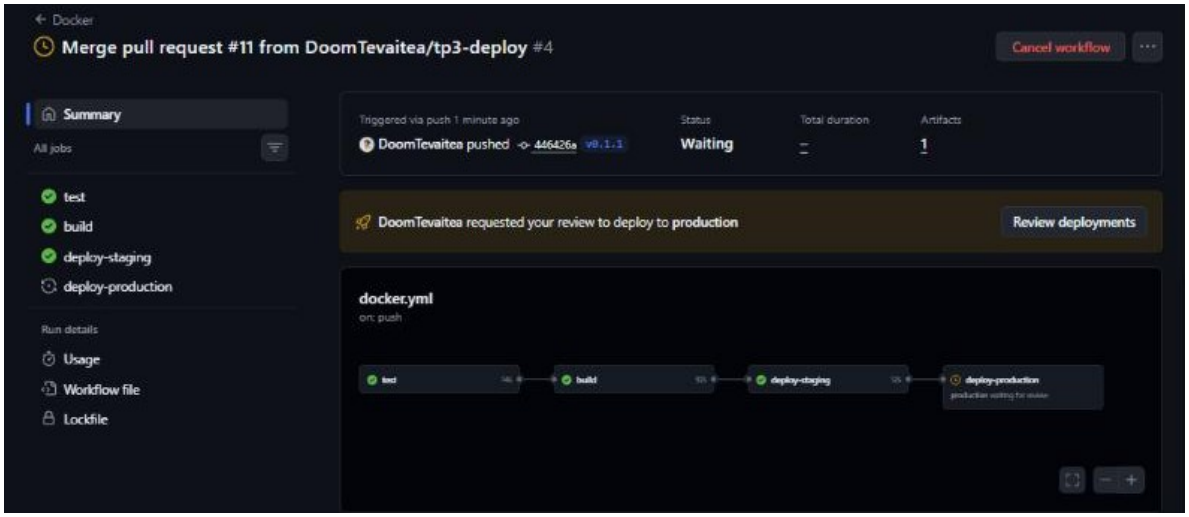
TP3-4 : la page du package `tasks-api` montre le tag `sha-7d8aa9c`.



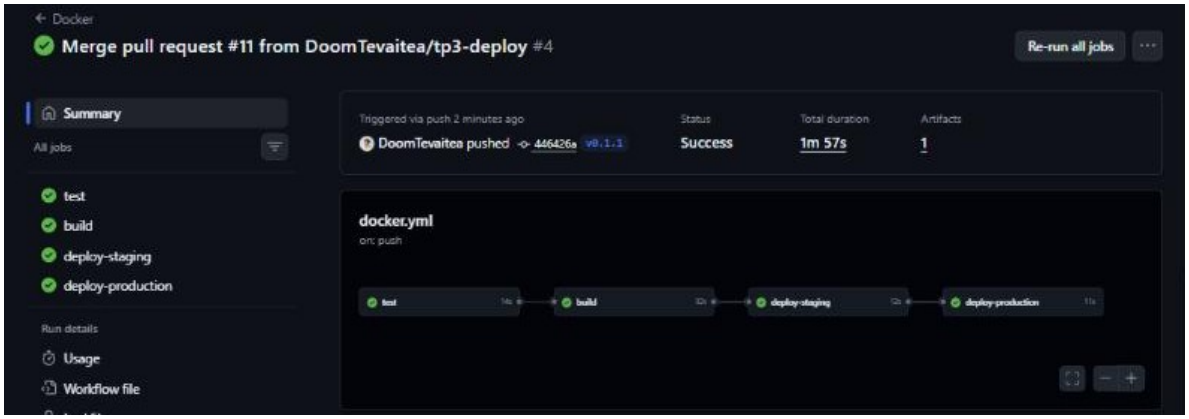
TP3-5 : la liste des versions contient les tags `0.1.0` et `sha-7d8aa9c`.

```
PS> docker pull ghcr.io/doomtevaitea/tasks-api:0.1.0
0.1.0: Pulling from doomtevaitea/tasks-api
a80f1242b924: Pull complete
acca7542dfc7: Already exists
3eb771151506: Already exists
6fc242d221a6: Already exists
f574edac59d9: Already exists
7f8fe038d998: Pull complete
9c09d95756f3: Pull complete
cfb6a168f318: Pull complete
ceaf4683bb88: Pull complete
44136fa355b3: Already exists
fe0451b549d2: Download complete
Digest: sha256:6470261015ffa35d85b5a70d81c7177262add4e13da1c1a8898fc14fe757e814
Status: Downloaded newer image for ghcr.io/doomtevaitea/tasks-api:0.1.0
ghcr.io/doomtevaitea/tasks-api:0.1.0
PS> docker run -d --rm --name tasks-ghcr -p 8000:8000 ghcr.io/doomtevaitea/tasks-api:0.1.0
f53aaa5298d34c6b590082ece5f5cb5d0a27710ac31a3eb46a31a64e902b9065
PS> curl.exe http://localhost:8000/version
{"version":"0.1.0"}
```

TP3-6 : l'image est téléchargée depuis GHCR, et `/version` répond `0.1.0`.



TP3-7 (attente) : le run `v0.1.1` a passé staging et attend l'approbation pour la production.



TP3-7 (terminé) : après l'approbation, tous les jobs sont verts.

```
PS> docker run --rm tasks-api:local id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

TP3-8 : la commande `id` montre que le conteneur tourne avec l'uid 10001 (`appuser`), donc pas en root.

Réponses aux questions

Différence entre une image et un conteneur ?

Une image est un modèle figé qui contient l'application et ses dépendances. Un conteneur est une exécution de cette image.

À quoi sert le .dockerignore ?

Il exclut des fichiers du contexte de construction, comme `.git` ou `.venv`. Le build est plus rapide, l'image plus petite, et on évite d'y copier des fichiers sensibles.

Quelle est la taille de l'image ? Quelles étapes sont CACHED, et pourquoi copier requirements.txt avant le code ?

L'image fait 201 Mo. Après une modification de `app/`, les étapes `WORKDIR`, `COPY requirements.txt` et `pip install` sont CACHED, et seule la copie du code recommence. Copier `requirements.txt` d'abord évite de réinstaller les dépendances à chaque changement de code.

Que gagne-t-on à décrire le lancement dans un fichier ?

La configuration est versionnée, partageable et rejouable avec une seule commande, sans retenir une longue ligne d'options.

Pourquoi build attend-il test ?

Pour ne pas publier une image construite à partir de code dont les tests échouent.

Pourquoi éviter le tag latest en production ?

Ce tag change de cible : on ne sait plus quelle version tourne, et le retour arrière devient incertain. Un numéro de version ou un SHA reste traçable.

Quel est l'avantage d'une image versionnée par rapport au code de ma machine ?

Tout le monde lance exactement le même contenu, avec le même environnement, et on peut revenir à une version précise.

À quoi sert un test de fumée ?

Il vérifie le strict minimum après un déploiement : l'application démarre et répond.

Pourquoi demander une approbation humaine avant la production ?

Une personne valide la mise en ligne auprès des vrais utilisateurs, ce qui évite un déploiement accidentel.

Pourquoi ne pas exécuter une application en root ?

En cas de faille, un attaquant aurait plus de droits dans le conteneur. Un utilisateur limité réduit les dégâts possibles.

Quelles bonnes pratiques tirer du cas Trivy ?

Épingler les actions par SHA, limiter les permissions et ne donner aux workflows que les secrets nécessaires.

Difficultés rencontrées

Le package était déjà public quand je l'ai vérifié : aucune modification de visibilité n'a été nécessaire.

La capture du run en attente devait être prise avant l'approbation, car cet état disparaît ensuite.

TP4 : GitOps

Objectif

Ce TP a déployé l'application avec le GitOps : le cluster suit le dépôt `tasks-config`, surveillé par Argo CD. Un tag sur le dépôt applicatif construit l'image, attend l'approbation, met à jour la configuration, puis Argo CD déploie la nouvelle version.

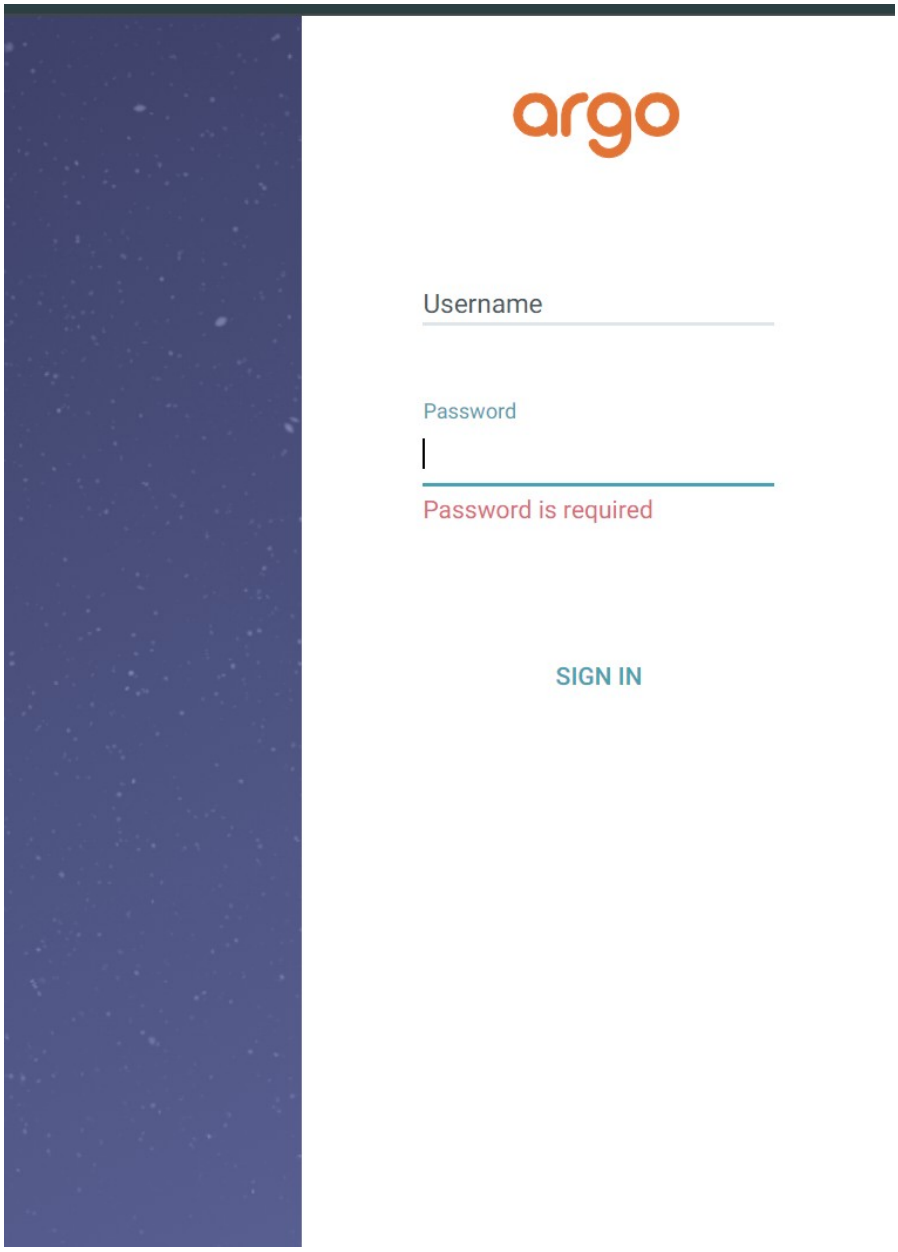
Captures d'écran

```
PS> kubectl get nodes
NAME                 STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   63m   v1.37.0
```

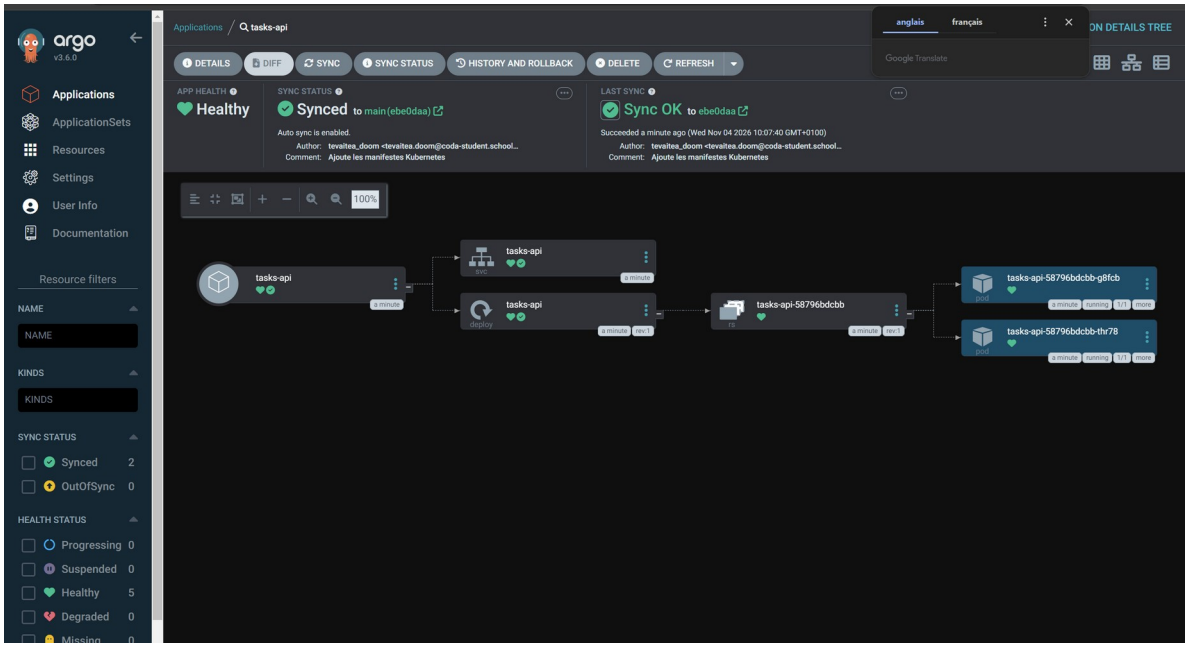
TP4-1 : `kubectl get nodes` montre le nœud `Ready`.

```
PS> kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          75s
argocd-applicationset-controller-56b6bc6dfc-jmdnr   1/1     Running   0          76s
argocd-dex-server-58f7b97876-7gfrp                  1/1     Running   0          76s
argocd-notifications-controller-7554d444c5-t9glz    1/1     Running   0          76s
argocd-redis-b459689dd-tc426                        1/1     Running   0          75s
argocd-repo-server-5bbf479748-pfcml                 1/1     Running   0          75s
argocd-server-f8cbdcd47-7cc26                       1/1     Running   0          75s
```

TP4-2 (pods) : tous les pods du namespace `argocd` sont `Running`.



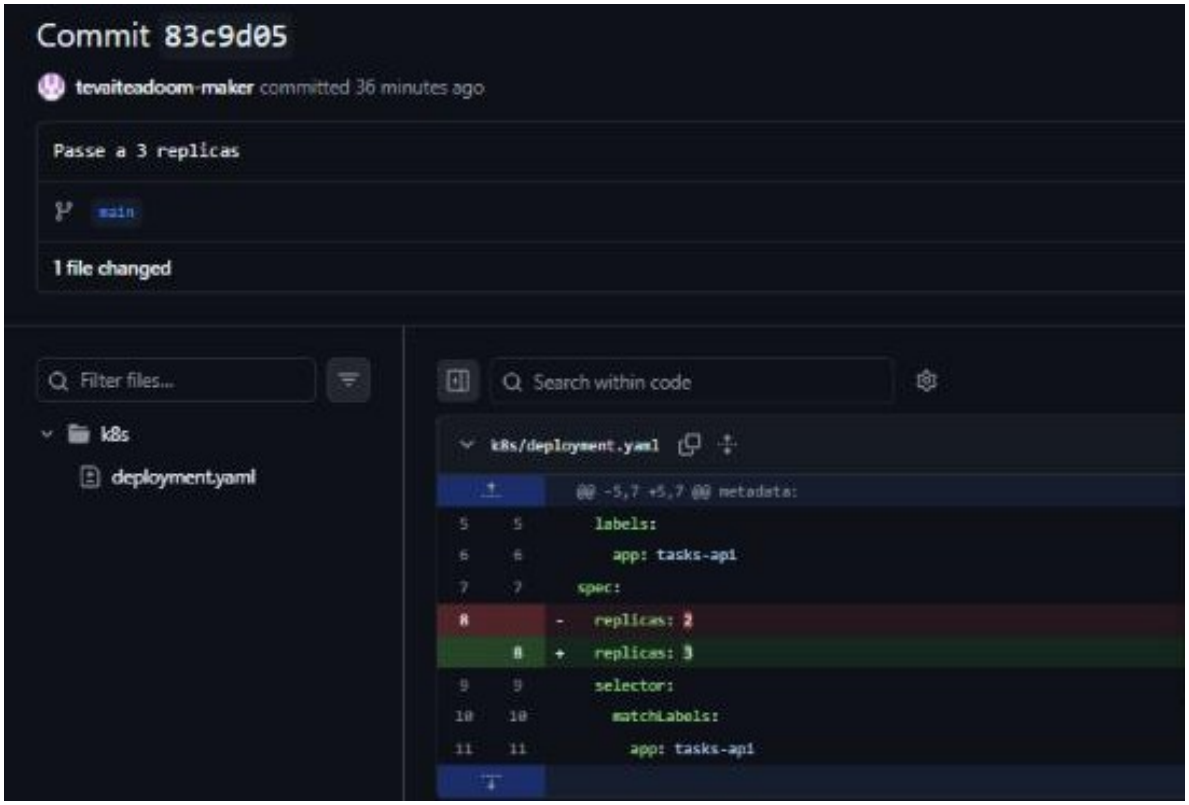
TP4-2 (connexion) : la page de connexion d'Argo CD s'affiche.



TP4-3 : l'application est `Synced` et `Healthy` dans Argo CD, avec ses 2 pods.

```
PS> curl.exe http://localhost:18081/version
{"version":"0.1.0"}
```

TP4-4 : `/version` répond `0.1.0` depuis le cluster, par une redirection sur le port 18081.



TP4-5 (commit) : le commit passe `replicas` de 2 à 3 dans `tasks-config`.

```
PS> curl.exe http://localhost:18081/version
{"version":"0.1.0"}
Capture enregistree : TP4-4

===== TP4 partie 3 : GitOps en action =====
[main 83c9d05] Passe a 3 replicas
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 402 bytes | 8.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DoomTevaitea/tasks-config.git
   ebe0daa..83c9d05  main -> main
Argo : Synced / Healthy, pods prets : 2, total : 2, image : ghcr.io/doomtevaitea/tasks-api:0.1.0
Argo : Synced / Healthy, pods prets : 3, total : 3, image : ghcr.io/doomtevaitea/tasks-api:0.1.0
API accessible sur le port 18081
```

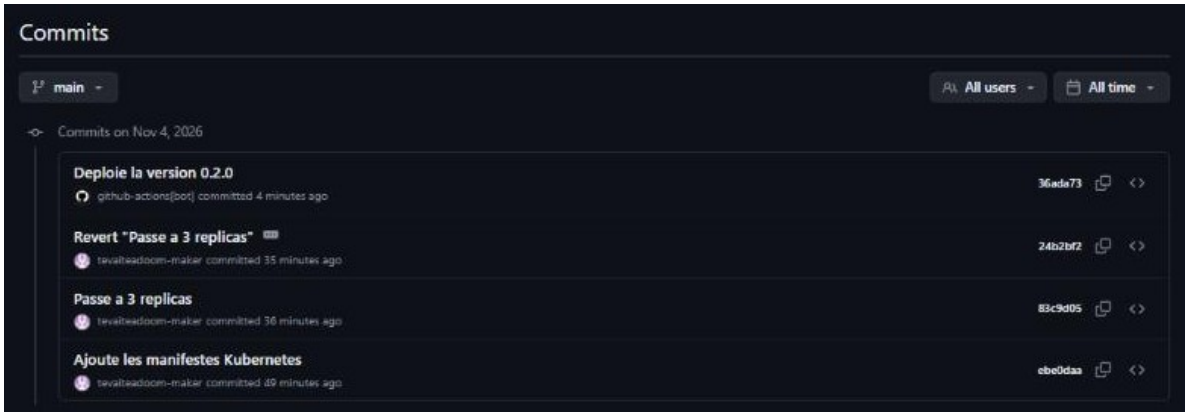
TP4-5 (pods) : le suivi dans le terminal montre l'état d'Argo CD : `Synced`, `Healthy` et 3 pods prêts.

```
PS> kubectl scale deployment tasks-api -n tasks --replicas=5
deployment.apps/tasks-api scaled
PS> kubectl get pods -n tasks
NAME                         READY   STATUS        RESTARTS   AGE
tasks-api-58796bdcbb-2sc7k   0/1     Terminating   0          1s
tasks-api-58796bdcbb-fmg4w   0/1     Terminating   0          1s
tasks-api-58796bdcbb-g8fcb   1/1     Running       0          13m
tasks-api-58796bdcbb-thr78   1/1     Running       0          13m
tasks-api-58796bdcbb-v6cpm   1/1     Running       0          47s
```

TP4-6 (scale) : juste après `kubectl scale` à 5, Argo CD a déjà commencé à arrêter les 2 pods en trop.

```
PS> kubectl get pods -n tasks
NAME                         READY   STATUS        RESTARTS   AGE
tasks-api-58796bdcbb-2sc7k   1/1     Terminating   0          7s
tasks-api-58796bdcbb-fmg4w   1/1     Terminating   0          7s
tasks-api-58796bdcbb-g8fcb   1/1     Running       0          13m
tasks-api-58796bdcbb-thr78   1/1     Running       0          13m
tasks-api-58796bdcbb-v6cpm   1/1     Running       0          53s
```

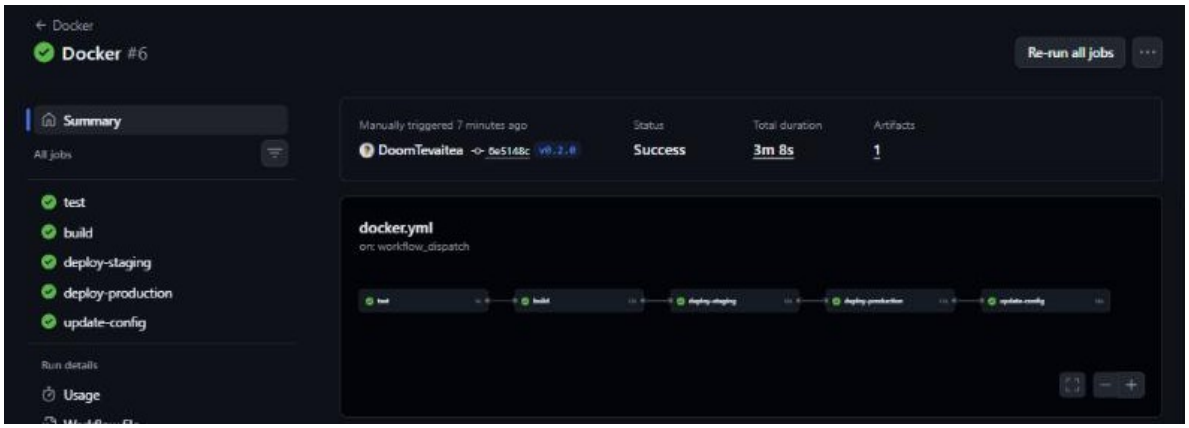
TP4-6 (correction) : quelques secondes plus tard, 3 pods restent en marche et les 2 autres se terminent.



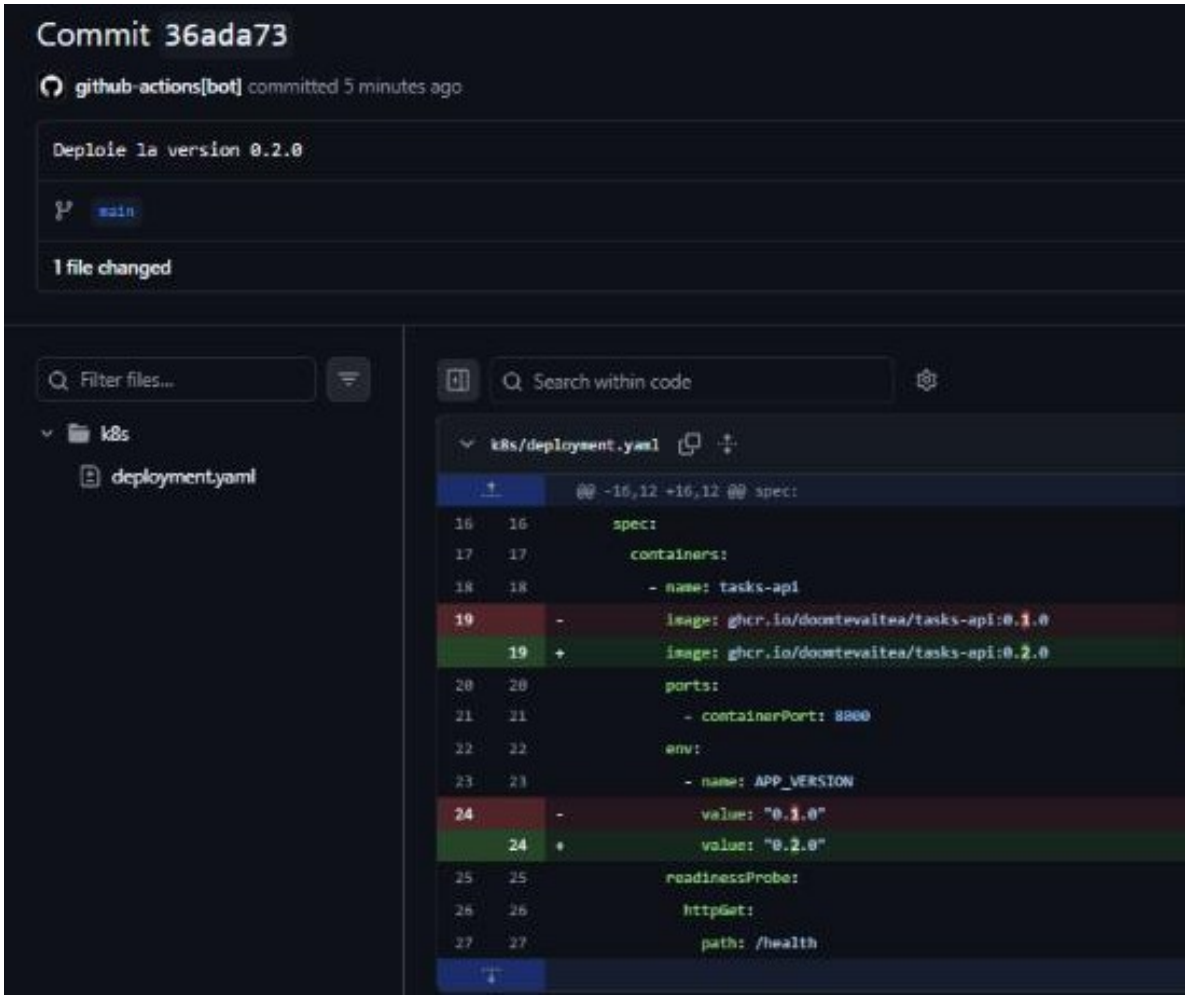
TP4-7 (historique) : l'historique de `tasks-config` contient le commit de `revert`.

```
PS> kubectl get pods -n tasks
NAME                         READY   STATUS        RESTARTS   AGE
tasks-api-58796bdcbb-2sc7k   1/1     Terminating   0          7s
tasks-api-58796bdcbb-fmg4w   1/1     Terminating   0          7s
tasks-api-58796bdcbb-g8fcb   1/1     Running       0          13m
tasks-api-58796bdcbb-thr78   1/1     Running       0          13m
tasks-api-58796bdcbb-v6cpm   1/1     Running       0          53s
Capture enregistree : TP4-6b
[main 24b2bf2] Revert "Passe a 3 replicas"
 Date: Wed Nov 4 10:21:09 2026 +0100
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 430 bytes | 33.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DoomTevaitea/tasks-config.git
   83c9d05..24b2bf2  main -> main
Argo : Synced / Healthy, pods prets : 3, total : 3, image : ghcr.io/doomtevaitea/tasks-api:0.1.0
Argo : Synced / Healthy, pods prets : 2, total : 2, image : ghcr.io/doomtevaitea/tasks-api:0.1.0
API accessible sur le port 18081
```

TP4-7 (pods) : après le `revert`, le suivi montre le retour à 2 pods prêts.



TP4-8 : le run `v0.2.0` est vert, `update-config` compris. Il a été lancé manuellement sur le tag.



TP4-9 (commit) : `github-actions[bot]` a modifié l'image et `APP_VERSION` pour passer en `0.2.0`.



TP4-9 (Argo CD) : Argo CD est synchronisé sur la configuration de la version `0.2.0`.

```
PS> kubectl get pods -n tasks
NAME                         READY   STATUS    RESTARTS   AGE
tasks-api-775d7cc868-52hkq   1/1     Running   0          10m
tasks-api-775d7cc868-5pwlc   1/1     Running   0          10m
PS> curl.exe http://localhost:18081/version
{"version":"0.2.0"}
```

TP4-10 : `/version` répond `0.2.0` dans le cluster.

Réponses aux questions

C'est quoi un pod ? Un deployment ? Pourquoi un namespace ?

Un pod est un ou quelques conteneurs qui tournent ensemble. Un deployment décrit le nombre de pods voulu et leur image, et Kubernetes le maintient. Un namespace range et isole les ressources.

Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?

Le code et le déploiement n'ont ni le même rythme ni les mêmes droits. L'historique de `tasks-config` devient le journal des déploiements.

Que signifie selfHeal ?

Argo CD corrige tout écart entre le cluster et Git. Une modification manuelle, comme le passage à 5 pods, est donc annulée.

Pourquoi le retour arrière est-il simple en GitOps ?

Un `git revert` crée un commit qui rétablit l'état précédent, puis Argo CD l'applique. L'opération reste tracée dans l'historique.

Résumez le chemin d'un git tag jusqu'au pod.

Le tag déclenche le pipeline, qui teste puis publie l'image. Le déploiement passe par staging, puis attend l'approbation pour la production. Le job `update-config` modifie alors `tasks-config`. Argo CD détecte ce commit et synchronise le cluster. De nouveaux pods démarrent avec la nouvelle image.

Qui pousse et qui tire ?

Le pipeline pousse l'image vers GHCR et le commit vers `tasks-config`. Argo CD tire la configuration depuis Git, et le cluster tire l'image depuis GHCR.

Quelle différence avec un déploiement CI/CD classique ?

Dans un déploiement classique, la CI se connecte au cluster et détient ses accès. Ici, l'agent dans le cluster lit Git, garde les accès et corrige les écarts en continu.

Difficultés rencontrées

Le port 8081 n'a pas répondu : j'ai accédé à l'API par le port 18081.

Le push du tag `v0.2.0` n'a pas déclenché de run : j'ai lancé le pipeline manuellement sur ce tag.

L'approbation par la ligne de commande a échoué : j'ai approuvé depuis la page du run.

Le passage à 3 pods et le retour à 2 sont prouvés par le suivi dans le terminal, car ces états n'ont pas été capturés dans Argo CD à temps.

Bilan

J'ai retenu qu'une CI fiable vérifie chaque changement, qu'une image versionnée rend les livraisons traçables, et qu'en GitOps on modifie Git plutôt que le cluster.

Le plus difficile a été de lire les erreurs de YAML et de suivre toute la chaîne, du tag jusqu'au pod.

Je referais autrement la prise des captures, en les préparant avant les étapes dont l'état disparaît ensuite.